ML for Mech Eng (CoEP)

Yogesh Haribhau Kulkarni

YHK

Outline

About Me

YHK

Yogesh Haribhau Kulkarni

Bio:

- 20+ years in CAD/Engineering software development
- Got Bachelors, Masters and Doctoral degrees in Mechanical Engineering (specialization: Geometric Modeling Algorithms).
- Currently doing Coaching in fields such as Data Science, Artificial Intelligence Machine-Deep Learning (ML/DL) and Natural Language Processing (NLP).
- Feel free to follow me at:
  - Github (github.com/yogeshhk)
  - LinkedIn (www.linkedin.com/in/yogeshkulkarni/)
  - Medium (yogeshharibhaukulkarni.medium.com)
  - Send email to yogeshkulkarni at yahoo dot com

Office Hours:
Saturdays, 2 to 5pm (IST); Free-Open to all; email for appointment.

YHK

Machine Learning Concepts

Machine Learning way of Solution, Mathematically

General Form: Exact Function

$Y = F(X_1, X_2, \ldots, X_p)$

- We have:
  - Observation of quantitative (numerical) response Y
  - Observation of p different predictors $X_1, X_2, \ldots, X_p$
- Find function F between Y and $X = X_1, X_2, \ldots, X_p$

Intuition

Think of Y as the thing you want to predict (say, a house's price) and $X_1, \ldots, X_p$ as the inputs you already know (size, location, age). F is simply the rule that turns those inputs into a prediction, if such an exact rule existed.

YHK

General Form: With Error Term

As one may not get exact function F, approximate it to f, thus introducing some error.

- So, general form: $Y = f(X) + \epsilon$
- f is unknown function of $X_1, X_2, \ldots, X_p$
- ϵ is a random error term, Independent of X, Has mean equal to zero
- Statistical learning refers to approaches for estimating f

Intuition

In practice we never find the perfect F from the previous slide, only an approximation f. The gap between what f predicts and reality is ϵ, unpredictable noise (measurement quirks, factors we didn't record). Statistical learning is just the toolbox for finding a good f from data.

YHK

Estimate f

- Not concerned if f is linear, quadratic, etc.
- We only care that our predictions are "near accurate".
- So, f often treated as a black box.
- The aim is of minimizing reducible error

YHK

Estimating f: Two Approaches

Most statistical learning methods classified as:

- Parametric
- Non-parametric

YHK

Estimating f: Parametric vs Non-Parametric

- Parametric:
  - Assume that the functional form, or shape, of f in linear in X, $f(X) = \sum_{i=1}^{p} \beta_i X_i$
  - This is a linear model, for p predictors $X = X_1, X_2, \ldots, X_p$
  - Model fitting involves estimating the parameters $\beta_0, \beta_1, \ldots, \beta_p$
  - Reduces the problem of estimating f to estimating a set of parameters
- Non-Parametric: No explicit assumptions about the functional form of f

Intuition

Parametric is like assuming the relationship is a straight line and just needing to find its slope and intercept, a handful of numbers (β's) to tune. Non-parametric makes no such assumption and lets the data itself trace out whatever curve fits best, more flexible, but it needs a lot more data to do so reliably.

YHK

Comparison

Parametric

- Only need to estimate set of parameters
- Bias such as linear model
- May not fit data well

Non-Parametric

- Potential of many shapes for f
- Lots of observations needed
- Complex models can overfit

YHK

Simple Example

- Linear regression is a simple and useful tool for predicting a quantitative response. The relationship between input variables $X = (X_1, X_2, \ldots X_p)$ and output variable Y takes the form: $Y \approx \beta_0 + \beta_1 X_1 + \ldots + \beta_p X_p + \epsilon$
- $\beta_0, \beta_1, \ldots \beta_p$ are the unknown coefficients (parameters) which we are trying to determine.
- The best coefficients will lead us to the best "fit", which can be found by minimizing the residual sum squares (RSS), or the sum of the difference between the actual ith value and the predicted ith value.
- $RSS = \sum_{i=1}^{n} e_i^2$, where $e_i = y_i - \hat{y}_i$ (actual minus predicted)

Intuition

This is just "fitting a line (or plane) through points." The β's are the slope and intercept, adjusting them tilts and shifts the line. RSS measures how far off the line's predictions are from the real points, squared so bigger misses count more, and the best-fit line is the one that makes this total miss as small as possible.

YHK

How to Find Best Fit: Matrix Form

Matrix Form:

- We can solve the closed-form equation for coefficient vector w: $w = (X^T X)^{-1} X^T Y$.
  - X: matrix of input data (each row is one data point, each column is one predictor); Y: column of output values
  - X^T: transpose of X (its rows and columns swapped)
  - $X^T X$: a square matrix summarizing how the predictors relate to each other
  - $(X^T X)^{-1}$: the "inverse" of that matrix, it undoes it, much like dividing by a number
  - $X^T Y$: summarizes how each predictor relates to the output Y
- This method is used for smaller matrices, since inverting a matrix is computationally expensive.

Intuition

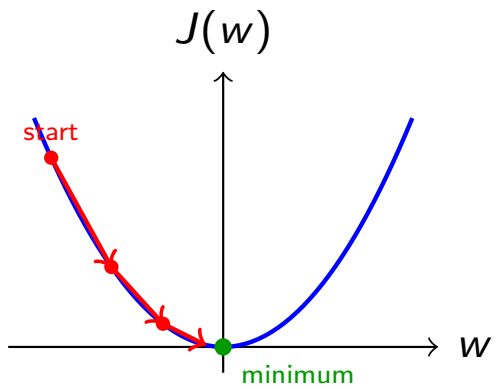
For linear regression there is actually a direct formula, plug in your data and this equation hands you the best-fit coefficients in one shot, no trial and error needed. The catch is that "inverting" a matrix gets expensive fast as the data grows, which is why this exact approach is only practical for smaller datasets.

YHK

How to Find Best Fit: Gradient Descent

Gradient Descent:

- First-order optimization algorithm.
- We can find the minimum of a convex function by starting at an arbitrary point and repeatedly take steps in the downward direction, which can be found by taking the negative direction of the gradient.
- After several iterations, we will eventually converge to the minimum. In our case, the minimum corresponds to the coefficients with the minimum error, or the best line of fit.
- The learning rate α determines the size of the steps we take in the downward direction.



Each step moves opposite to the gradient (steepest descent); α sets the step size.

YHK

Quick Check: The Math Framework

Think About It

The general form $Y = f(X) + \epsilon$ includes an irreducible error term ϵ that no model, however good, can eliminate. If you had a perfect model that estimated f exactly, would your predictions ever be 100% accurate?

YHK

Quick Check: The Math Framework

Answer

No, even with a perfect estimate of f, the ϵ term (random noise independent of X) still contributes to prediction error. This is why statistical learning aims to minimize the reducible error (the gap between $\hat{f}$ and the true f), not to eliminate all error entirely.

YHK

Cross Validation

Background: Imp Terms

- **Learning algorithm:** identifies a model that best fits the relationships between inputs and outputs.
- **Training set:** consists of records with features and with/without class labels
- **Test set:** consists of records with only features, class labels to be computed.

YHK

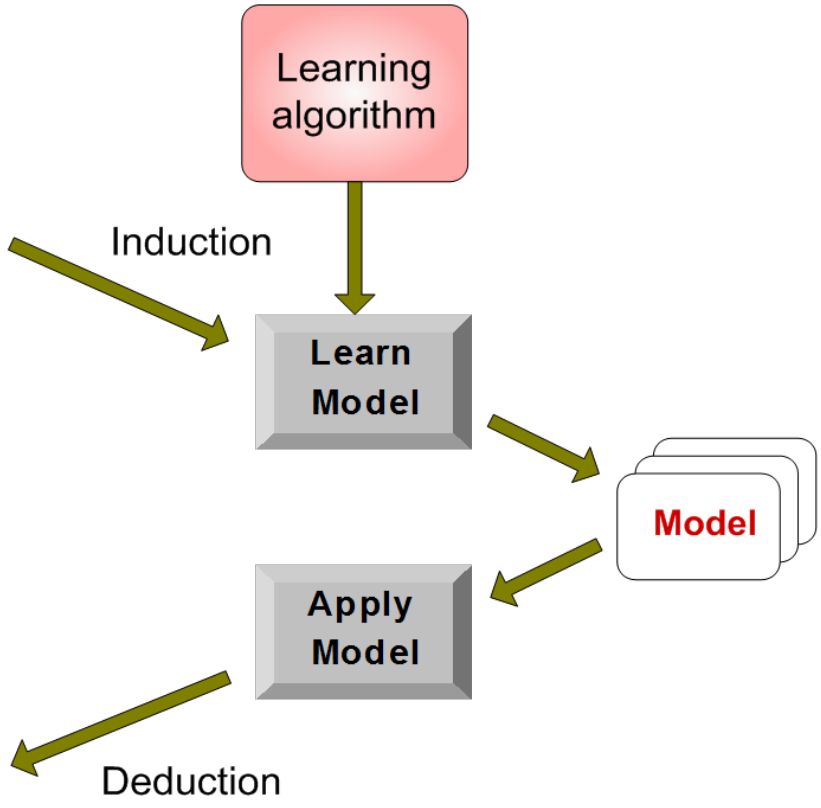
Training - Testing

Tid	Attrib1	Attrib2	Attrib3	Class
1	Yes	Large	125K	No
2	No	Medium	100K	No
3	No	Small	70K	No
4	Yes	Medium	120K	No
5	No	Large	95K	Yes
6	No	Medium	60K	No
7	Yes	Large	220K	No
8	No	Small	85K	Yes
9	No	Medium	75K	No
10	No	Small	90K	Yes

Training Set

Tid	Attrib1	Attrib2	Attrib3	Class
11	No	Small	55K	?
12	Yes	Medium	80K	?
13	Yes	Large	110K	?
14	No	Small	95K	?
15	No	Large	67K	?

Test Set



(Reference: Data Mining Classification - Gun Ho Lee)

YHK

Cross Validation: Measuring Performance

- Labeled training data is finite.
- The model built needs to be validated.
- So, training data itself is split
  - Use one part for model building: Training Data.
  - Use the other part: Testing Data. (actually called as Validation Data, but within Cross Validation process, it can be called as Testing data. Otherwise Testing data is the one which does not have output values. Validation data has output values for comparison.)

YHK

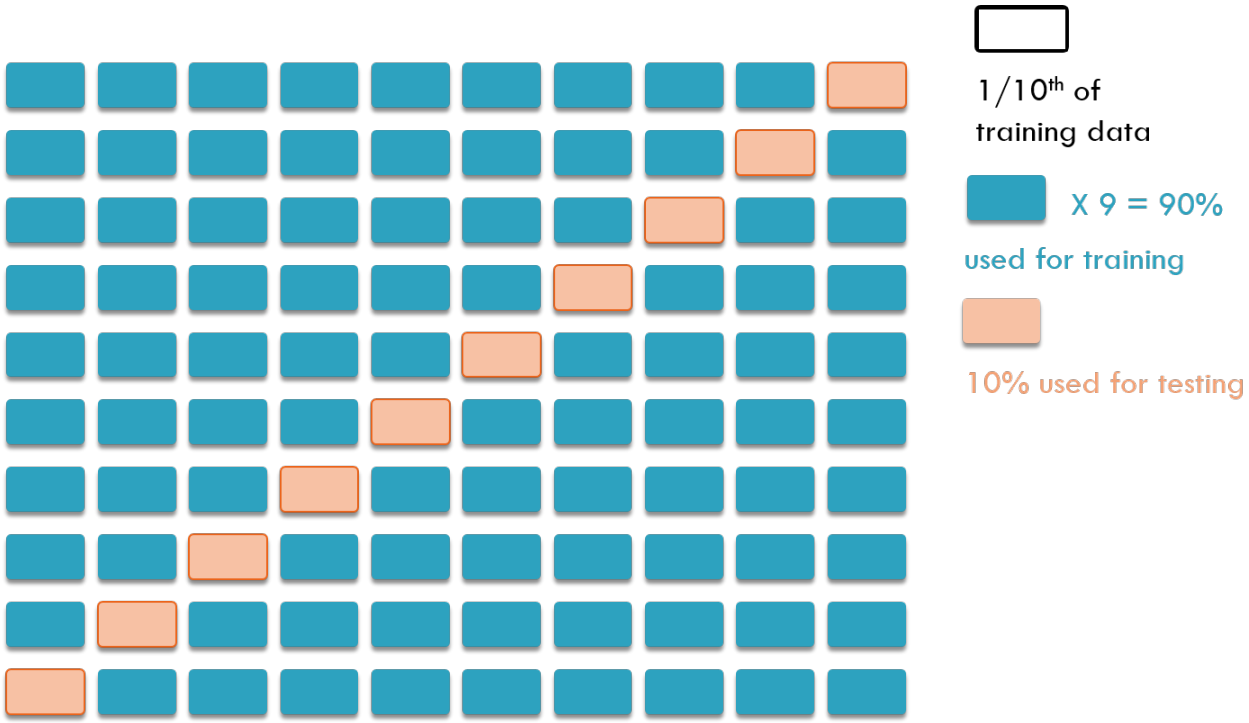
Cross Validation

How to divide dataset into Training, Validation, Testing sets?

- Problems?
  - Validation set should "represent" the Training set.
  - "Lucky split" is possible: Difficult instances were chosen for the training set, Easy instances put into the testing set
- Multiple evaluations using different portions of data for training and validating/testing
- k-fold Cross Validation

YHK

K-Fold



Cross Validation Error Estimate: $= \frac{1}{k} \sum Error_i$

Note: Here weights of all 10 models are neither averaged nor the last weights are taken as final model. [PTO]

YHK

Quick Check: Cross Validation

Think About It

If you split your data into training and testing sets just once, and your model gets high accuracy, does that guarantee it will perform well on new, unseen data?

YHK

Quick Check: Cross Validation

Answer

Not necessarily: a single split can be a "lucky split" where the testing set happens to be easy, or the training set happens to cover the harder cases well by chance. K-fold cross-validation guards against this by evaluating the model across multiple different splits and averaging the results, giving a more reliable estimate of true performance.

YHK

Machine Learning Evaluation

Evaluation

Given Actuals and Predictions, how to find out how good the performance was?

- Accuracy: Misclassification Rate
- Confusion Matrix
- Precision, Recall, F1

YHK

Evaluation Metrics: Accuracy

Accuracy (Regression): Mean Squared Error

Model Evaluation on Test Set (Regression) - Mean Squared Error

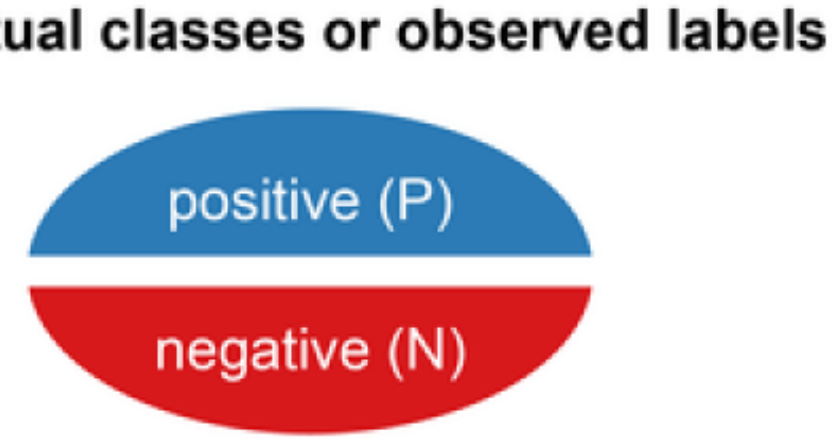
- Mean Squared Error: measuring the "quality of fit"
- Will be small if the predicted responses are very close to the true responses $MSE = \frac{1}{n}\sum(y_i - f(x_i))^2$

YHK

Evaluation Metrics: Confusion Matrix

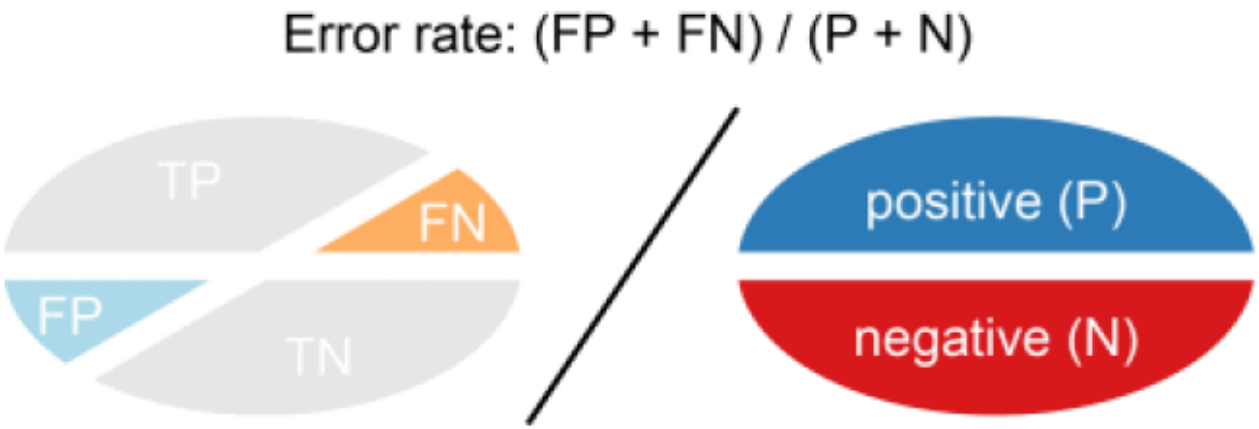
Evaluation Metrics

There are predicted labels (Positive / negative) as well as actual labels (Positive / negative) .



In binary classification, a test dataset has two labels; positive and negative.

Error rate (ERR)

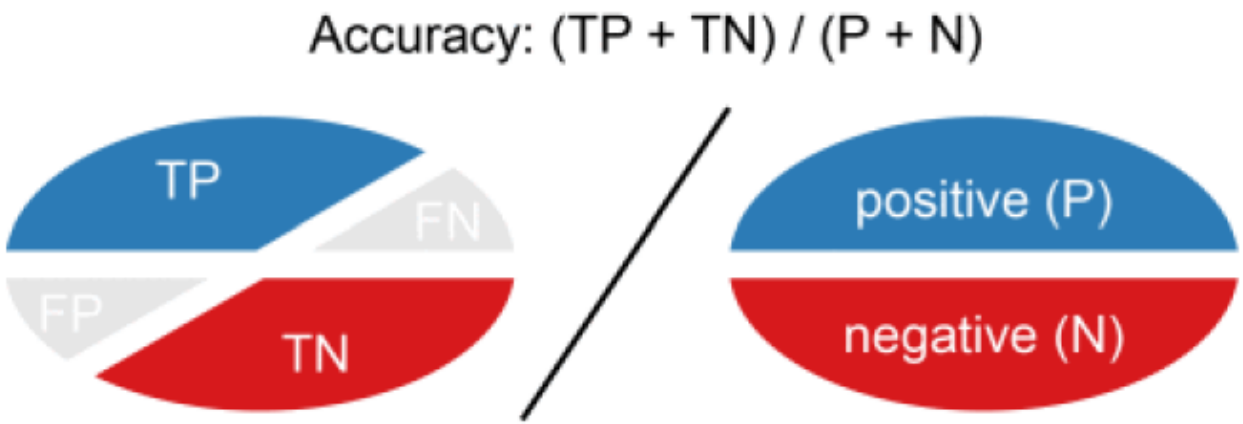


It is calculated as the number of all incorrect predictions divided by the total number of the dataset. The best error rate is 0.0, whereas the worst is 1.

$$ERR = \frac{FP+FN}{TP+TN+FN+FP} = \frac{FP+FN}{P+N}$$

Accuracy (Classification)

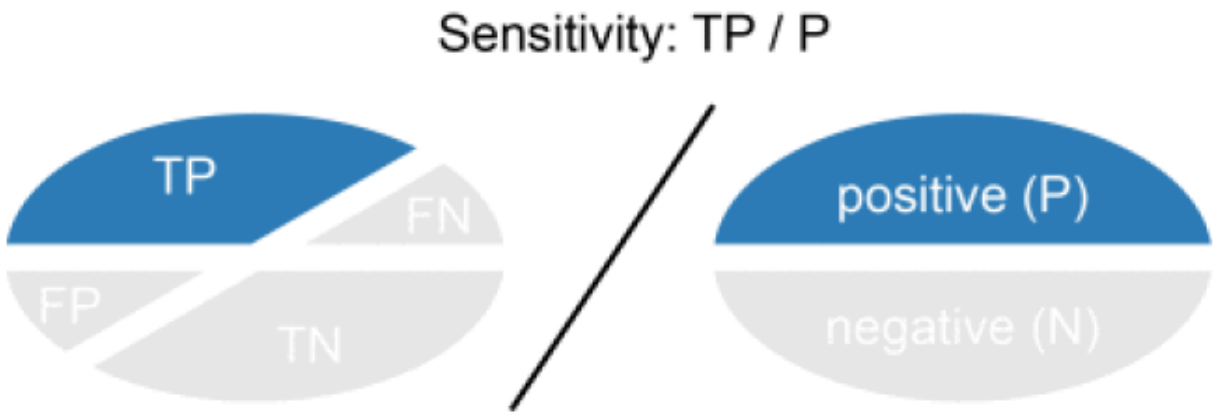
It is calculated as the number of all correct predictions divided by the total number of the dataset. The best accuracy is 1.0, whereas the worst is 0.0. It can also be calculated by $1 - ERR$.



$$Accuracy = \frac{TP+TN}{P+N}$$

Recall (Sensitivity or True positive rate)

Sensitivity (SN) is calculated as the number of correct positive predictions divided by the total number of positives. It is also called recall (REC) or true positive rate (TPR). The best sensitivity is 1.0, whereas the worst is 0.0.

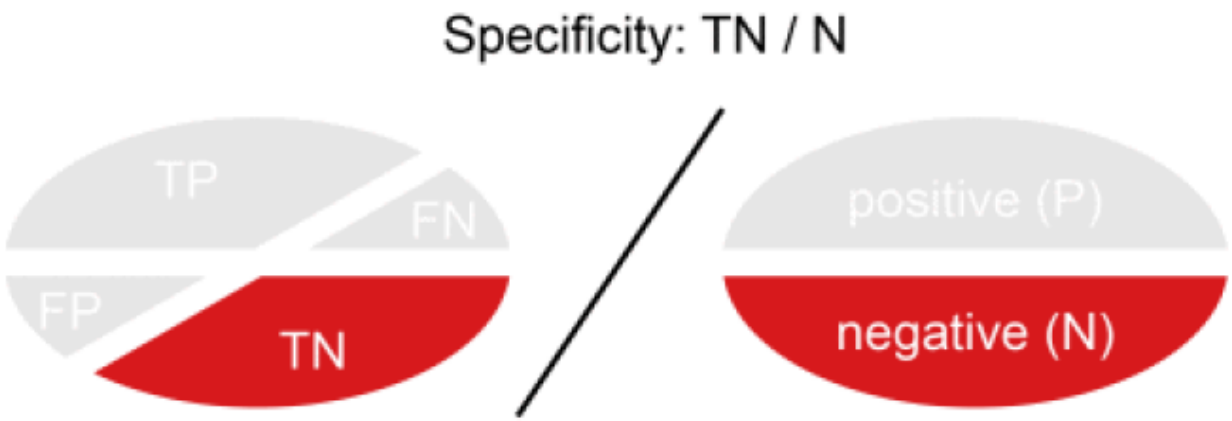


$$SN = \frac{TP}{TP+FN} = \frac{TP}{P}$$

YHK

Specificity (True negative rate)

Specificity (SP) is calculated as the number of correct negative predictions divided by the total number of negatives. It is also called true negative rate (TNR). The best specificity is 1.0, whereas the worst is 0.0.



$$SP = \frac{TN}{TN+FP} = \frac{TN}{N}$$

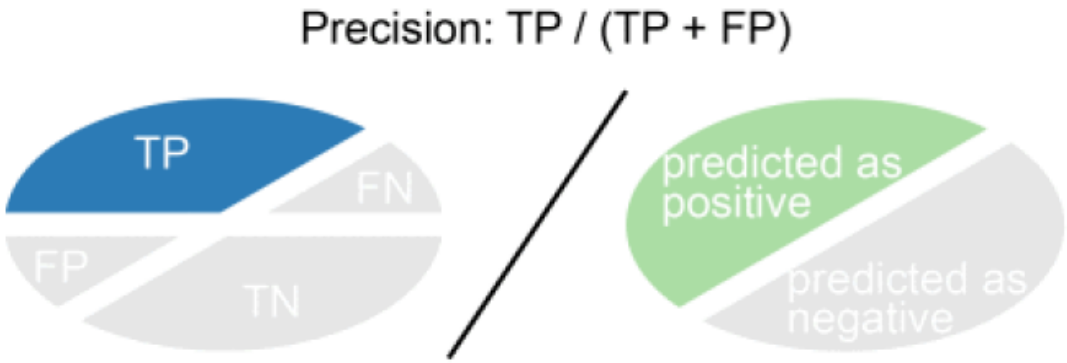
Sensitivity vs Specificity

- For sensitivity we look at only the Positives group (all predicted positive); the rest fall into the negatives group.
- For highly sensitive test, 100% sensitive, within the Negatives group, there is no mistake.
- Meaning anyone who has tested negative, they surely don't have disease.
- All are truly negative. Meaning FN are 0. Recall thus can be made 1 by marking all positive. So there is no one in the Negatives group. So no FN. But thats not good.
- But still its not a perfect test, because within Positives, you may have some FPs.
- Similarly 100% Specific test: We will send all negative tested guys to a different group. Within them we had FN guys as well. The Positives were perfect. All of them had disease. $FP = 0$. So those who tested positive surely had the disease.
- In reality there is no perfect test!!

YHK

Precision (Positive predictive value)

Precision (PREC) is calculated as the number of correct positive predictions divided by the total number of positive predictions. It is also called positive predictive value (PPV). The best precision is 1.0, whereas the worst is 0.0.



$$PREC = \frac{TP}{TP+FP}$$

Precision: Intuition

Fishing for a specific fish

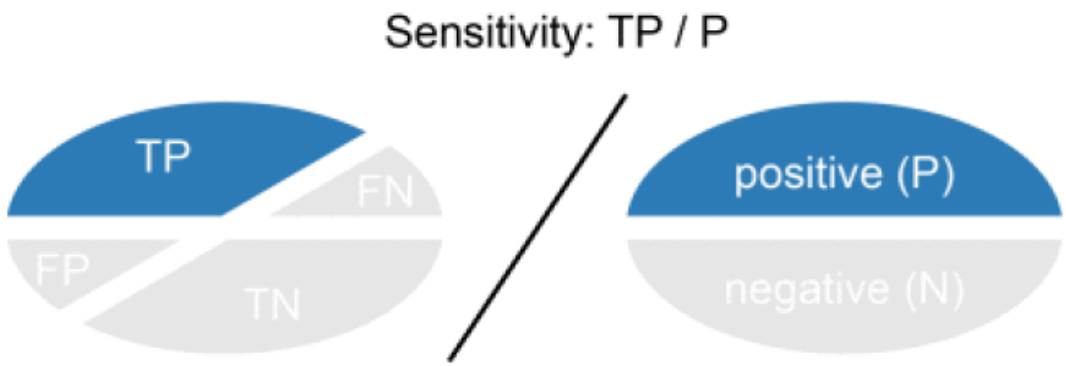
You are out on the lake, but you only want rainbow trout. At the end of the day you look into your bucket and ask: of the fish I actually kept, what fraction were really rainbow trout?

- That fraction is precision. The denominator is what you **claimed**, not what exists.
- It says nothing about the trout still swimming in the lake; those never entered the calculation.
- So precision punishes False Positives: every carp in your bucket drags it down.
- Use a finer, more selective net (a stricter threshold) and precision goes up.

YHK

Recall (Coverage of actual positives)

Recall (REC) is calculated as the number of correct positive predictions divided by the total number of **actual** positives. This is the same quantity as sensitivity seen earlier; it is worth restating here, now that precision is on the table, because the two are always read as a pair.



$$REC = \frac{TP}{TP+FN} = \frac{TP}{P}$$

YHK

Recall: Intuition

Casting the net

Same lake, different question: of all the fish that were in the lake to begin with, what fraction ended up in my net?

- That fraction is recall. The denominator is what **exists**, not what you claimed.
- Fish that got away are False Negatives, so recall punishes misses.
- Cast a wider net (a relaxed threshold) and recall goes up; in the limit, net the whole lake and recall is 1.0.
- But that same wide net hauls in boots and weeds, so precision falls. Hence the trade-off.

YHK

Precision vs Recall: One Net, Two Questions

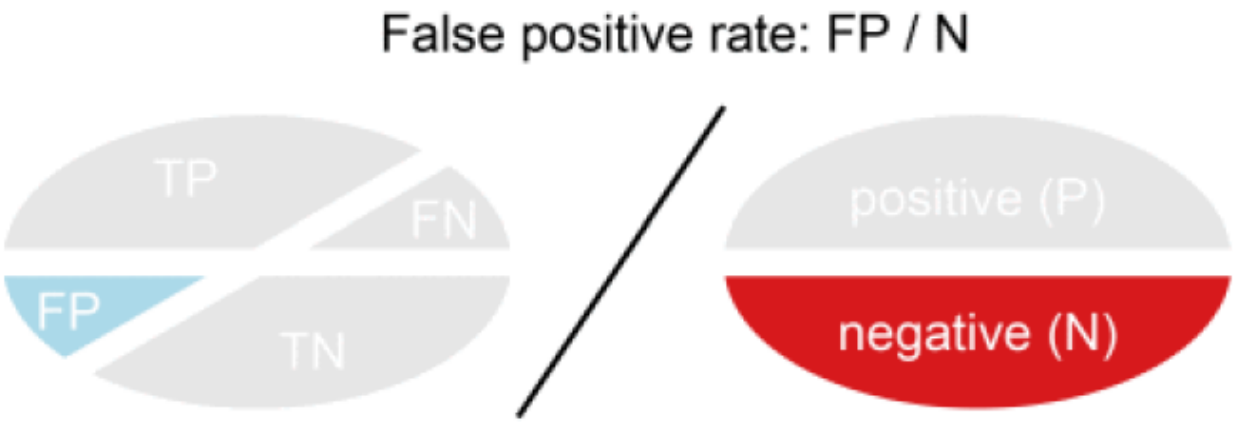
	Precision	**Recall**
Question	Of what I caught, how much was right?	Of what was there, how much did I catch?
Formula	$\frac{TP}{TP+FP}$	$\frac{TP}{TP+FN}$
Denominator	What the model claimed	What actually exists
Punishes	False Positives (false alarms)	False Negatives (misses)
Improved by	Stricter threshold, finer net	Relaxed threshold, wider net
Care when	A wrong flag is expensive: spam filter, arrest, costly recall of parts	A miss is expensive: cancer screening, fraud, crack detection

Note the two metrics have different denominators. That is exactly why they cannot both be pushed up freely, and why averaging them needs care (see F1).

YHK

False positive rate

False positive rate (FPR) is calculated as the number of incorrect positive predictions divided by the total number of negatives. The best false positive rate is 0.0 whereas the worst is 1.0. It can also be calculated as 1 - specificity.



$$FPR = \frac{FP}{TN+FP} = 1 - SP$$

YHK

Example

		True condition	
	Total population	Condition positive	Condition negative
Predicted condition	Predicted condition positive	**True positive**	**False positive**, Type I error
	Predicted condition negative	**False negative**, Type II error	**True negative**

Type I Error

Type I Error is equivalent to False Positive (FP).

- Test is Positive but actually disease is not there.
- A Fire alarm going off when in fact there is no fire.

This kind of error is synonymous to “believing a lie” or “a false alarm”.

YHK

Type II Error

Type II Error is equivalent to False Negative (FN).

- Test is Negative but actually disease is there.
- A Fire breaking out and the fire alarm does not ring.

This kind of error is synonymous to “failing to believe a truth” or “a miss”.

YHK

Evaluation Metrics: Precision and Recall

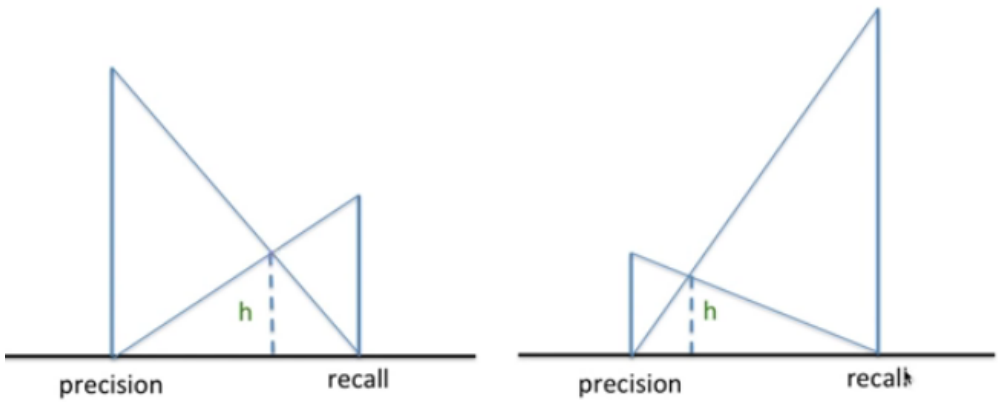
Precision and Recall

- Precision: fraction of records that are positive in the set of records that classifier predicted as positive
- Interpretation: If the prediction is very stringent then getting all of them correct is likely, so more precise. Less (no) mis-predictions (FP) happens.
- Recall: fraction of positive records that are correctly predicted by classifier, out of all actual.
- Interpretation: Criteria is relaxed so that maximum catch is made. Less (no) mis-classification (FN) happens.

YHK

F1 Measure

- F-measure: combines precision and recall into a single metric, using the harmonic mean
- Harmonic Mean of two numbers tends to be closer to the smaller of two numbers, so the only way F1 is high is for both precision and recall to be high. $F_1 = \frac{2rp}{(r+p)} = \frac{2 \times TP}{(2 \times TP + FP + FN)}$



h is half the harmonic mean

Even if anyone is very high, we get balanced F1 score
(Ref: Performance measure on multiclass classification - Minsuk Heo)

YHK

Why Harmonic Mean and not the Others?

Three models, three ways of averaging their precision and recall:

P	R	Arithmetic	Geometric	Harmonic (F1)
0.5	0.5	0.50	0.50	0.50
0.9	0.1	0.50	0.30	0.18
1.0	0.0	0.50	0.00	0.00

- The arithmetic mean scores all three rows at 0.50. It cannot tell a genuinely balanced model from one that is useless on one side.
- Row 3 is the giveaway: a model with zero recall still scores 0.50. Averaging additively lets a strong metric pay for a failed one.
- The harmonic mean collapses towards the weaker number, so it is 0.18 and 0.00 for the last two rows. F1 is high only when **both** are high.

YHK

Why Harmonic Mean: The Reasoning

- Precision and recall are **rates with different denominators** ($TP + FP$ versus $TP + FN$). The harmonic mean is the standard way to average rates, because it averages the denominators rather than the ratios.
- It is deliberately pessimistic. Being the reciprocal of the average reciprocal, one small value blows up its reciprocal and drags the whole result down.
- That pessimism is the point. It blocks the cheap way to game the metric: flag every sample positive and recall becomes 1.0 while precision drops to the base rate. Arithmetic mean rewards that trick, F1 does not.
- The geometric mean also punishes imbalance and reaches 0 in the extreme, but harmonic punishes harder in the middle (0.18 versus 0.30 above), so it is the convention.
- Order holds throughout: harmonic $\leq$ geometric $\leq$ arithmetic, with equality only when precision and recall are equal.

YHK

Which One Should You Optimize?

- Precision addresses False Positives and Recall addresses False Negatives, so the choice comes down to which of the two errors is more expensive for your problem.
- If you are hunting something rare (skewed) like Cancer detection or Fraud, you are typically concerned with FN, because the test must not wrongly clear someone who actually has the disease. A missed case is far costlier than a follow-up test on a false alarm. Use Recall.
- When a false alarm itself is expensive or disruptive, such as a spam filter deleting a genuine mail, or scrapping a good part on the shop floor, use Precision.
- When both errors matter and you need a single number to compare models, use F1.

YHK

Precision and Recall: Tug of War

Precision and Recall trade-off

- Typically to increase precision for a given model implies lowering recall, though this depends on the precision-recall curve of your model.
- Generally, if you want higher precision you need to restrict the positive predictions to those with highest certainty in your model, which means predicting fewer positives overall (which, in turn, usually results in lower recall).
- If you want to maintain the same level of recall while improving precision, you will need a better classifier.

YHK

Precision and Recall trade-off : Summary

- Intuitively, if you cast a wider net (meaning your classifier threshold is relaxed), you will detect more relevant documents/positive cases (higher recall)
- But you will also get more false alarms (lower precision).
- If you classify everything in the positive category, you have 100% recall (no FN here), a bad precision because some of them could be wrong, so there will be many FPs, making precision lower.
- If you adjust threshold strict so as to have good precision, you will have lower making of positives and mark many actually positive values as negative, falsely. Making more FNs, thus lower recall.

YHK

Quick Check: Evaluation Metrics

Think About It

A hospital builds a test to flag a rare but serious disease. Should they optimize for high precision or high recall, and why?

YHK

Quick Check: Evaluation Metrics

Answer

High recall: missing a genuine case (a false negative) is far more costly here than a false alarm, since a missed diagnosis can be life-threatening while a false positive just triggers a follow-up test. It's the same precision-recall trade-off from the slides, but which side you favor depends entirely on which error type costs more for the specific problem.

YHK

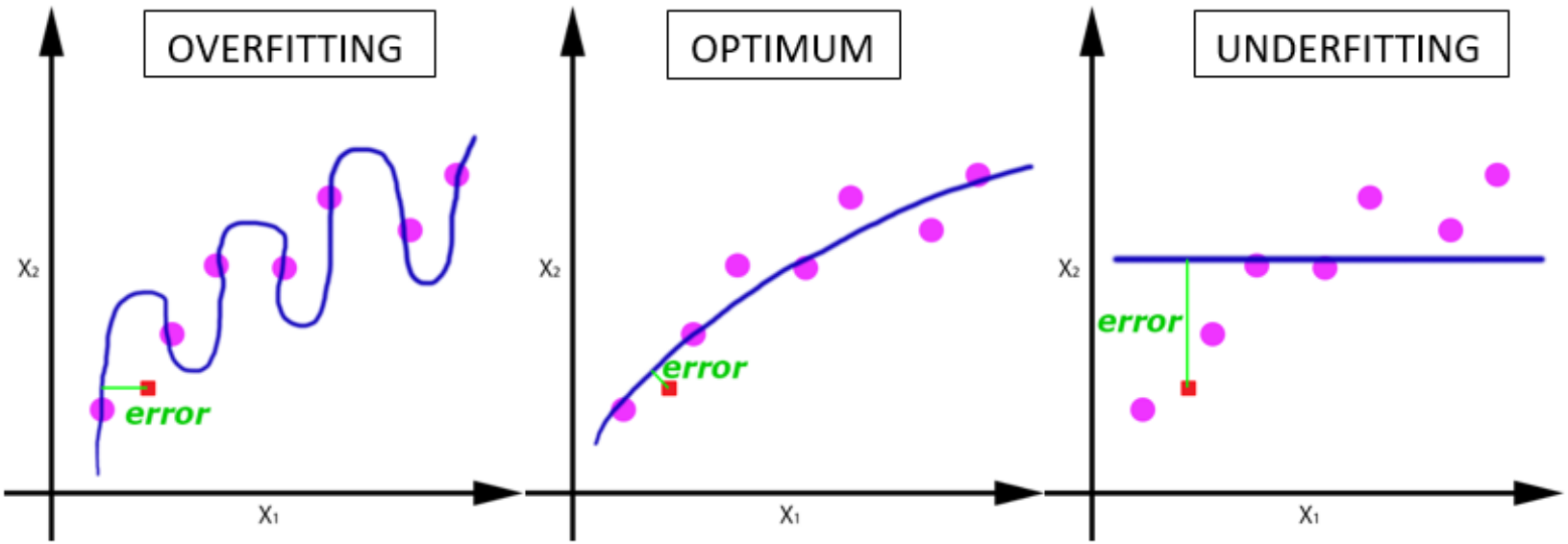
Machine Learning Model Generalization

Generalization in Machine Learning

- Generalization refers to how well the concepts learned by a machine learning model apply to specific examples not seen by the model when it was learning.
- The goal of a good machine learning model is to generalize well from the training data to any data from the problem domain.
- This allows us to make predictions in the future on data the model has never seen
- In statistics, a fit refers to how well you approximate a target function.

YHK

Under-fitting vs Over-fitting



(Reference: What is Machine Learning? - An Introduction- itdadao)

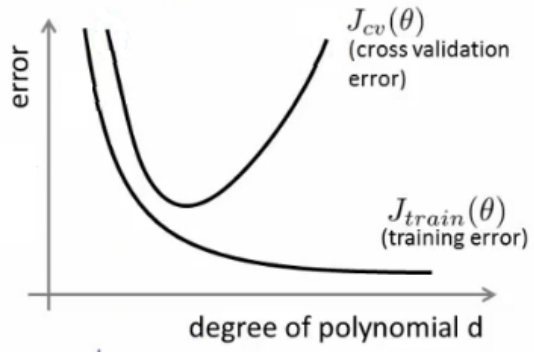
Over-fitting

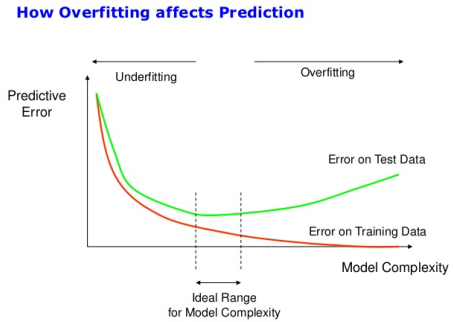
- Over-fitting: occurs when model "memorizes" training data
- Model does not generalize to the overall problem
- This is bad! We wish to avoid over-fitting
- Techniques: Regularization (Penalty) and Drop Out (Neural Network)

YHK

Detection

Roughly speaking, over-fitting typically occurs when the ratio $\frac{ComplexityOfTheModel}{TrainingSize}$ is too high.





- If your data is in two dimensions, you have 10000 points in the training set and the model is a line, you are likely to under-fit.
- If your data is in two dimensions, you have 10 points in the training set and the model is 100-degree polynomial, you are likely to over-fit.

(Ref: Why Is Over-fitting Bad in Machine Learning? - StackOverflow)

YHK

Solution

- In many texts, “iterations” is used on X axis, which is easy to imagine in case of Neural Networks, where each epoch refines model by fitting better question/weights.
- In case of Machine Learning, there no iterations. How to detect Over-fitting in case of Polynomial Regression?
- Initially when, polynomial equation to start with is simple, it has not fit the data yet, so its under fitting phase. Losses are HIGH for both, training as well as testing (validation)
- As more levels get added in tree or more degrees in polynomial regression, under-fitting reduces and losses come down.
- At one point, training loss still reduces but validation loss starts jumping up. Thats over-fitting. Model is fitting training data TOO tightly and is OFF the validation data.

(Ref: Why Is Over-fitting Bad in Machine Learning? - StackOverflow)

YHK

Regularization: Intuition

- In regularization, apart from cost function, weighted sum of parameters and features is added
- For the features which need to be suppressed, their corresponding coefficient is made large. So their cost component increases
- As the Cost needs to be minimized, to reduce these weighted sum, corresponding feature values are reduced,
- So their importances gets lowered dramatically.

YHK

Regularization: Formal Definition

- Instead of simply aiming to minimize loss (empirical risk minimization): $minimize(Loss(Data|Model))$
- We'll now minimize loss+complexity, which is called structural risk minimization: $minimize(Loss(Data|Model) + complexity(Model))$

YHK

Under-fitting

- Under-fitting: occurs when model cannot fit training data well
- Model does generalize to the overall problem but not that well
- This is also bad! We wish to avoid under-fitting
- Techniques: more features

YHK

Quick Check: Generalization

Think About It

A model gets 99% accuracy on training data but only 60% on test data. Is this under-fitting or over-fitting, and what's one way to fix it?

YHK

Quick Check: Generalization

Answer

Over-fitting: the model has memorized the training data (near-perfect training score) but hasn't learned patterns that generalize, so it fails on unseen data. Regularization (penalizing complexity) or reducing model complexity (e.g. fewer polynomial degrees or a shallower tree) are standard fixes.

YHK

Bias and Variance

Types of Errors

The prediction error can be broken down into:

- Noise Error
- Bias Error
- Variance Error

$$\text{error}(X) = \text{noise}(X) + \text{bias}(X) + \text{variance}(X)$$

Intuition

Noise is the irreducible randomness in the data itself, nothing can fix it. Bias is error from a model too simple to capture the true pattern. Variance is error from a model too sensitive to the specific training data it happened to see. The next few slides unpack bias and variance in detail.

YHK

Bias

- Bias is the simplifying assumption made by a model to make the target function easier to learn.
- A high bias makes it fast to learn and easier to understand but is generally less flexible. In turn, it has lower predictive performance on complex problems.
- Low Bias: Suggests less assumptions about the form of the target function.
- High-Bias: Suggests more assumptions about the form of the target function.

YHK

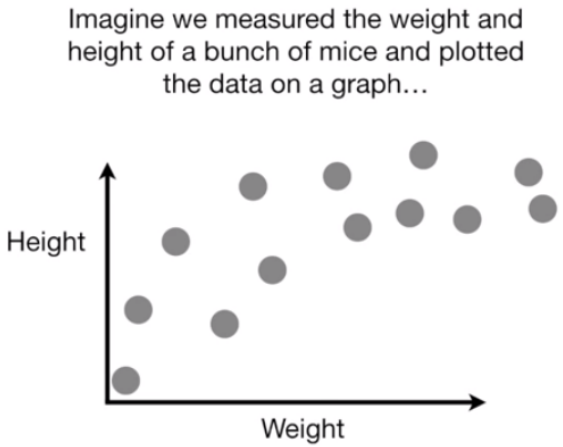
Variance

- Variance is the amount that the estimate of the target function will change if different training data was used.
- Ideally, it should not change too much from one training dataset to the next, meaning that the algorithm is good at picking out the hidden underlying mapping between the inputs and the output variables.
- Low Variance: Suggests small changes to the estimate of the target function with changes to the training dataset.
- High Variance: Suggests large changes to the estimate of the target function with changes to the training dataset.

YHK

Intuition: Example

- We have collected past observations of Mice's Heights and Weights.
- Clearly,the relationship is sort of curved.
- Wish to predict Height given Weight.
- Different algorithms will have different underlying structures to fit this regression data.

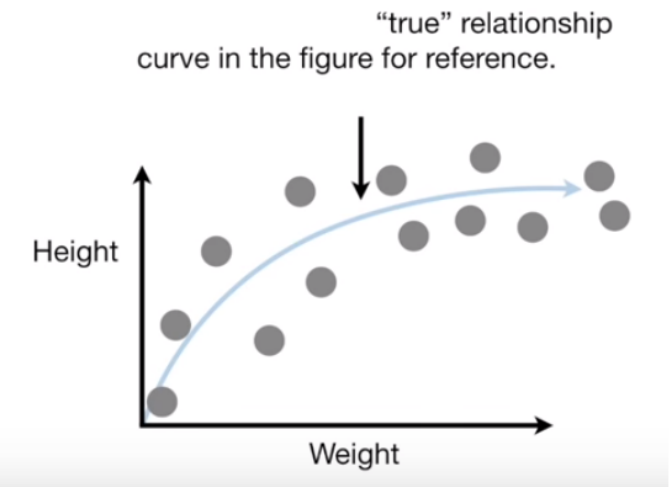


(Ref: Machine Learning Fundamentals: Bias and Variance -StatQuest with Josh Starmer)

YHK

Intuition: True Relationship

- We have collected past observations of Mice's Heights and Weights.
- Clearly,the relationship is sort of curved.
- Different algorithms will have different underlying structures to fit this regression data.

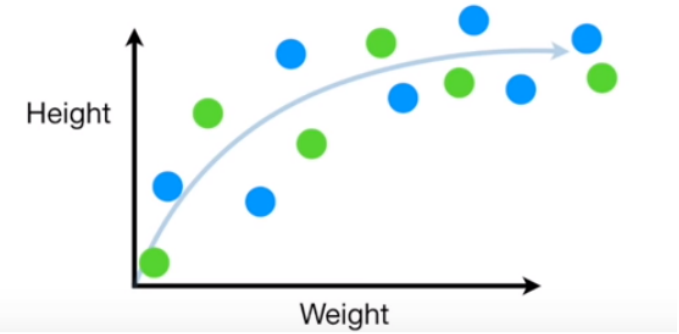


(Ref: Machine Learning Fundamentals: Bias and Variance -StatQuest with Josh Starmer)

YHK

Intuition: Machine Learning

- Split data into training and testing (validation)
- Blue for training
- Green for testing

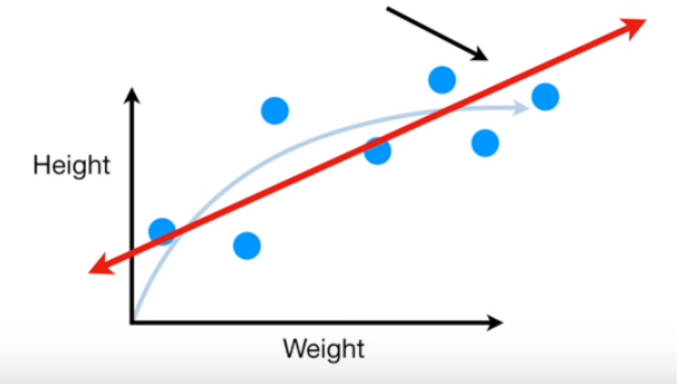


(Ref: Machine Learning Fundamentals: Bias and Variance -StatQuest with Josh Starmer)

YHK

Intuition: Linear Regression

- Fitting Linear Regression model on the training data (blue dots)
- It fits a line, whatever the underlying data is.
- But it finds best line that minimizes Least Square error.
- Anyway, it can not bend, so said to have HIGH bias. It is not very accurate.
- Inability of Machine Learning model to capture the TRUE relationship is called as **Bias**.

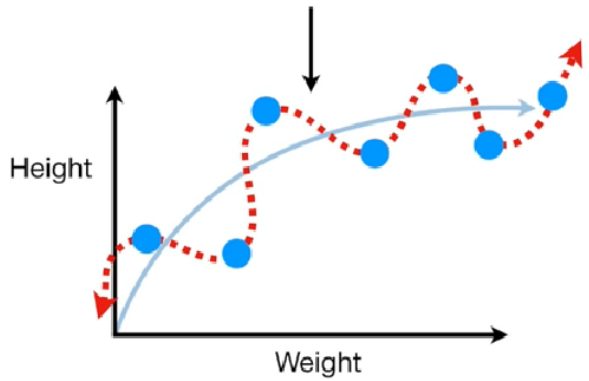


(Ref: Machine Learning Fundamentals: Bias and Variance -StatQuest with Josh Starmer)

YHK

Intuition: Non-Linear (Tree) Regression

- Some other Machine Learning model can give high degree and flexible model.
- As it passes through points, its very accurate (as there is just no error)
- Being flexible, it has ability to represent true relationship, so very little bias.

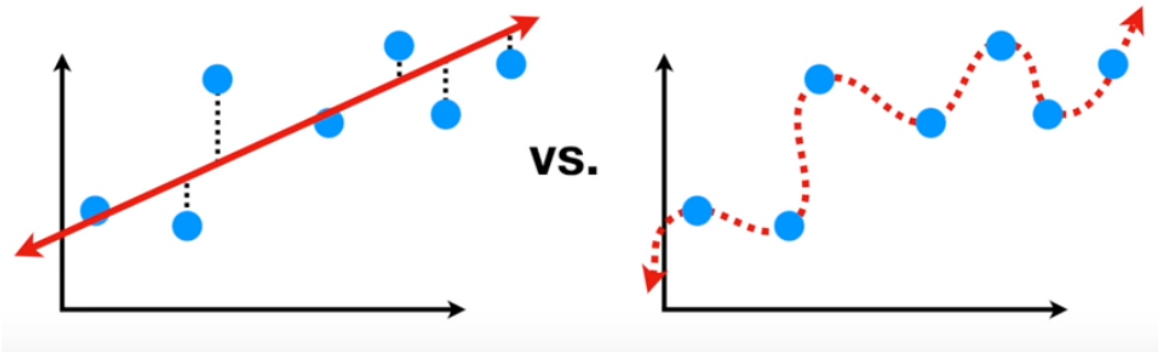


(Ref: Machine Learning Fundamentals: Bias and Variance -StatQuest with Josh Starmer)

YHK

Intuition: Training set Comparison

- Linear model has error on Training set. High Bias.
- Non-linear model has almost no error on Training set. Low Bias.

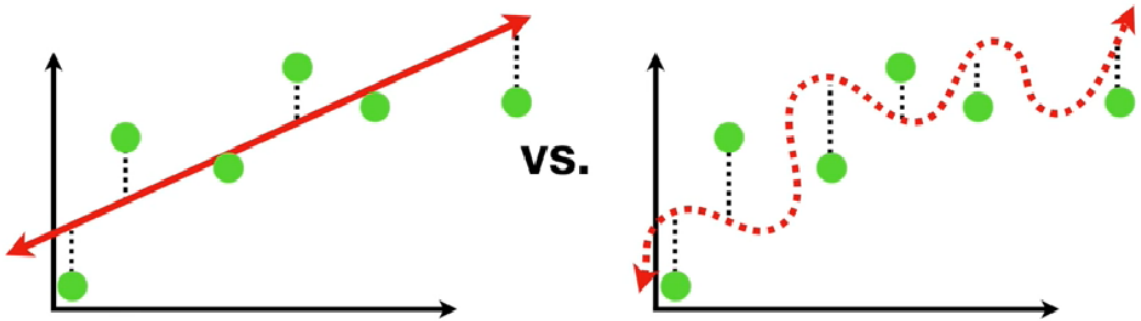


(Ref: Machine Learning Fundamentals: Bias and Variance -StatQuest with Josh Starmer)

YHK

Intuition: Testing set Comparison

- When same model is applied on the testing set, how does it fair?
- Which model looks better? Linear or Non-Linear?
- Linear is not accurate, but the Non-linear is far too bad.
- This is called as **Variance**.
- Difference in fits on different datasets is the Variance.
- Linear model has less Variance and Non Linear model has High Variance.

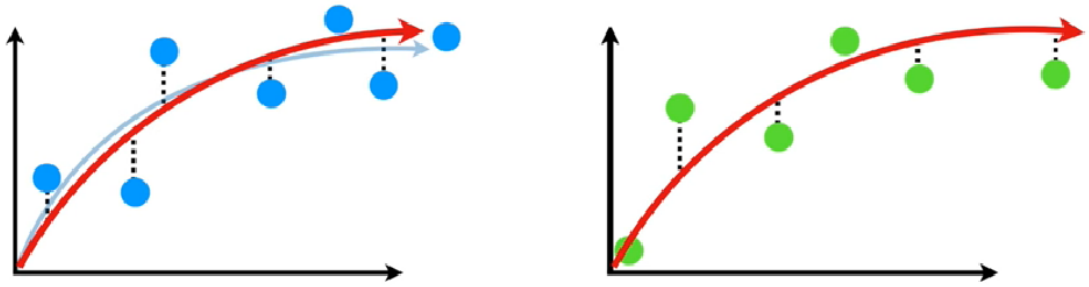


(Ref: Machine Learning Fundamentals: Bias and Variance -StatQuest with Josh Starmer)

YHK

Intuition: Ideal

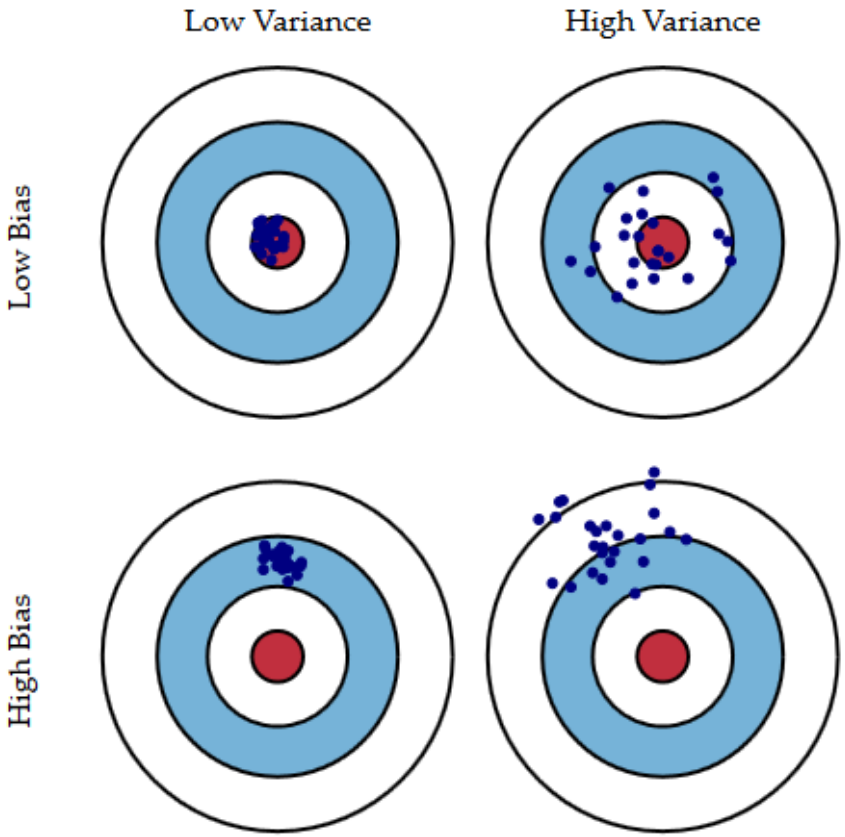
- Linear model gives good predictions but not great predictions, but its consistent.
- Non linear model gives great predictions but not consistent many a times.
- Ideal model captures underlying true relationship. So, low Bias and low Variance.
- It is done by finding a Sweet spot between simple model and complex model.
- 3 such methods of finding sweet spot are : regularization, bagging and boosting.



(Ref: Machine Learning Fundamentals: Bias and Variance -StatQuest with Josh Starmer)

YHK

Bias vs Variations



http://scott.fortmann-roe.com/docs/BiasVariance.html

YHK

Bias Variance Trade-Off

What is the trade-off?

- Generally, as the model becomes more computational (e.g. when the number of free parameters grows), the variance (dispersion) of the estimate also increases, but bias decreases.
- Due to the fact that the training set is memorized completely instead of generalizing, small changes lead to unexpected results (over-fitting).
- On the other side, if the model is too weak, it will not be able to learn the pattern, resulting in learning something different that is offset with respect to the right solution.

YHK

But Why is there a trade-off?

- Low bias algos tend to be more complex and flexible e.g. Decision Tree (non-linear) so that it can fit the data well.
- But if the algo is too flexible, it will fit each training data set differently, and hence have high variance.
- A key characteristic of many supervised learning methods is a built-in way to control the bias-variance trade-off either automatically or by providing a special parameter that the data scientist can adjust.

YHK

Quick Check: Bias-Variance Trade-Off

Think About It

A very simple linear model and a very complex model both have low error on the training data. Does that tell you which one will generalize better to new data?

YHK

Quick Check: Bias-Variance Trade-Off

Answer

Not by itself: low training error alone can't distinguish a well-fit model from an overfit one. The complex model might have low bias but high variance (it may just be memorizing noise), so you need to check performance on held-out validation/test data, which is exactly why cross-validation exists, before trusting either model's training-set performance.

YHK

Summary

- Statistical learning reveals hidden data relationships.
- Relationships between dependent and independent data.

YHK

Finally

All models are wrong; but some are useful - George E P Box

YHK

Thanks . . .

- Office Hours: Saturdays, 3 to 5 pm (IST); Free-Open to all; email for appointment to yogeshkulkarni at yahoo dot com
- Call + 9 1 9 8 9 0 2 5 1 4 0 6

(https://www.linkedin.com/in/yogeshkulkarni/)

(https://medium.com/@yogeshharibhaukulkarni)

(https://www.github.com/yogeshhk/)

YHK